

myFFT2 函数学习报告

1. 函数整体作用

`myFFT2` 的作用是：用 FFT 加速计算二维连续傅里叶变换在指定输出网格上的离散采样结果。

它计算的数学形式是：

$$U_c^1(X, Y) = \iint U_c(x_0, y_0) e^{-i2\pi(x_0X + y_0Y)} dx_0 dy_0$$

其中：

- `Ud` 是原函数 $U_c(x_0, y_0)$ 在输入平面上的离散采样；
- `xmin, xmax, ymin, ymax` 是输入平面的坐标范围；
- `Xmin, Xmax, Ymin, Ymax` 是输出频域或像面坐标范围；
- `M, N` 是输出平面的采样点数；
- `Ud1` 是傅里叶变换结果在输出平面上的离散采样。

该函数不是简单地做 `fft2(Ud)`，而是实现了一个可以指定输入、输出坐标范围的二维傅里叶变换采样算法。

2. 与夫琅禾费衍射的关系

夫琅禾费衍射公式的核心结论是：远场衍射场等于孔径函数的傅里叶变换。

标量形式为：

$$U(x, y) = \frac{e^{ikz}}{i\lambda z} e^{i\frac{k}{2z}(x^2 + y^2)} \iint U_0(x_0, y_0) e^{-i\frac{2\pi}{\lambda z}(xx_0 + yy_0)} dx_0 dy_0$$

其中：

$$k = \frac{2\pi}{\lambda}$$

令：

$$X = \frac{x}{\lambda z}, \quad Y = \frac{y}{\lambda z}$$

则积分部分变成：

$$\iint U_0(x_0, y_0) e^{-i2\pi(x_0X + y_0Y)} dx_0 dy_0$$

这正是 `myFFT2` 中要计算的傅里叶变换形式。

因此，在光学语境下，代码中的 `X`、`Y` 可以理解为空间频率坐标。如果需要对应实际观察屏坐标，需要使用：

$$x = \lambda z X, \quad y = \lambda z Y$$

3. 输入和输出网格

设 `Ud` 的大小为：

$$n \times m$$

其中：

- `m = size(Ud, 2)`：输入平面 x 方向采样点数，也就是列数；
- `n = size(Ud, 1)`：输入平面 y 方向采样点数，也就是行数。

输入平面坐标为：

$$x_j = x_{\min} + jk_x, \quad j = 0, 1, \dots, m-1$$

$$y_l = y_{\min} + lk_y, \quad l = 0, 1, \dots, n-1$$

其中：

$$k_x = \frac{x_{\max} - x_{\min}}{m - 1}$$

$$k_y = \frac{y_{\max} - y_{\min}}{n - 1}$$

代码中对应：

matlab

```
1 | kx = (xmax - xmin) / (m - 1);  
2 | ky = (ymax - ymin) / (n - 1);
```

输出平面坐标为：

$$X_p = X_{\min} + pk_X, \quad p = 0, 1, \dots, M-1$$

$$Y_q = Y_{\min} + qk_Y, \quad q = 0, 1, \dots, N-1$$

其中：

$$k_X = \frac{X_{\max} - X_{\min}}{M-1}$$

$$k_Y = \frac{Y_{\max} - Y_{\min}}{N-1}$$

代码中对应：

matlab

```
1 kX = (Xmax - Xmin) / (M - 1);  
2 kY = (Ymax - Ymin) / (N - 1);
```

4. 直接计算的形式

二维连续傅里叶变换离散化后为：

$$Ud1_{q,p} \approx \sum_{l=0}^{n-1} \sum_{j=0}^{m-1} Ud_{l,j} e^{-i2\pi(x_j X_p + y_l Y_q)} k_x k_y$$

其中：

$$x_j = x_{\min} + jk_x$$

$$y_l = y_{\min} + lk_y$$

$$X_p = X_{\min} + pk_X$$

$$Y_q = Y_{\min} + qk_Y$$

如果直接双重循环计算，每一个输出点都要对所有输入点求和，计算量很大。

`myFFT2` 的核心技巧是：把这个求和转化成卷积，再利用 FFT 快速计算。

5. 关键思想：把傅里叶核改写成卷积形式

先只看一维 x 方向。

需要计算：

$$U_p^1 = \sum_{j=0}^{m-1} U_j e^{-i2\pi x_j X_p} k_x$$

代入：

$$x_j = x_{\min} + jk_x$$

$$X_p = X_{\min} + pk_X$$

得到：

$$e^{-i2\pi x_j X_p} = e^{-i2\pi(x_{\min} + jk_x)(X_{\min} + pk_X)}$$

展开指数中的乘积：

$$(x_{\min} + jk_x)(X_{\min} + pk_X) = x_{\min}X_{\min} + x_{\min}pk_X + jk_xX_{\min} + jpk_xk_X$$

其中最麻烦的是交叉项：

$$jpk_xk_X$$

因为它同时含有输入索引 j 和输出索引 p ，不能直接变成卷积。

使用恒等式：

$$2jp = j^2 + p^2 - (p - j)^2$$

所以：

$$e^{-i2\pi jpk_xk_X} = e^{-i\pi k_xk_Xj^2} e^{-i\pi k_xk_Xp^2} e^{i\pi k_xk_X(p-j)^2}$$

这样就把交叉项拆成三部分：

1. 只和输入索引 j 有关的部分；
2. 只和输出索引 p 有关的部分；
3. 只和索引差 $p - j$ 有关的部分。

其中第三部分正好具有卷积形式。

二维情况同理，只是同时包含 x 和 y 两个方向。

6. U1 是怎么来的

代码中：

matlab

```
1 U1 = Ud .* exp(-1j*pi*(kx * kX * ix.^2 + ky * kY * iy.^2 + 2 * kx * Xmin *  
    ix + 2 * ky * Ymin * iy));
```

U1 是把原始输入数据 Ud 乘上输入端相位因子之后得到的。

数学形式为：

$$U1_{l,j} = Ud_{l,j} e^{-i\pi[k_x k_X j^2 + k_y k_Y l^2 + 2k_x X_{\min} j + 2k_y Y_{\min} l]}$$

其中：

- j 是输入平面 x 方向索引；
- l 是输入平面 y 方向索引。

这部分包含的是傅里叶核中只和输入索引有关的相位项。

从一维角度看，U1 对应：

$$U1_j = U_j e^{-i\pi k_x k_X j^2} e^{-i2\pi k_x X_{\min} j}$$

所以 U1 可以理解为：

$$U1 = \text{原始数据} \times \text{输入端 chirp 相位}$$

7. U2 是怎么来的

代码中：

matlab

```
1 IX = [Ix, -m:-1];
2 IY = [Iy, -n:-1];
3 [IX, IY] = meshgrid(IX, IY);
4 U2 = exp(1j*pi*(kx * kX * IX.^2 + ky * kY * IY.^2));
```

U2 是卷积核，来自拆分出来的索引差部分：

$$e^{i\pi[k_x k_X (p-j)^2 + k_y k_Y (q-l)^2]}$$

因此：

$$U2_{q-l, p-j} = e^{i\pi[k_x k_X (p-j)^2 + k_y k_Y (q-l)^2]}$$

其中：

- $p - j$ 是 x 方向的输出索引与输入索引之差；
- $q - l$ 是 y 方向的输出索引与输入索引之差。

U2 可以理解为：

$$U2 = \text{只依赖索引差的 chirp 卷积核}$$

| 8. 如何理解 $p - j$

在卷积中，一维线性卷积定义为：

$$C_r = \sum_j A_j B_{r-j}$$

其中：

- r 是输出结果的索引；
- j 是第一个序列的索引；
- $r - j$ 是第二个序列在当前输出位置下对应的索引。

这可以理解成：计算第 r 个输出点时，所有满足

$$j + s = r$$

的下标组合都会相乘求和。

因为：

$$s = r - j$$

所以卷积公式写成：

$$C_r = \sum_j A_j B_{r-j}$$

在 `myFFT2` 中：

$$r \leftrightarrow p$$

$$A_j \leftrightarrow U1_j$$

$$B_{r-j} \leftrightarrow U2_{p-j}$$

所以：

$$p - j = \text{输出点编号} - \text{输入点编号}$$

它不是实际物理坐标差，而是数学推导中为了构造卷积而出现的索引差。

9. 从“翻转和平移”理解卷积

线性卷积也可以理解为：先把一个序列镜像翻转，再平移，然后与另一个序列逐点相乘求和。

一维卷积：

$$C_r = \sum_j A_j B_{r-j}$$

其中：

1. 原来的 B_j 镜像翻转后变成 B_{-j} ；
2. 再平移 r ，变成 B_{r-j} ；
3. 然后与 A_j 相乘并对 j 求和。

因此，`U2_{p-j}` 就可以理解成：

卷积核 $U2$ 先镜像翻转，再平移到输出位置 p 后对应输入点 j 的取值

二维卷积为：

$$C_{q,p} = \sum_l \sum_j A_{l,j} B_{q-l,p-j}$$

对应两个方向的翻转和平移：

- x 方向： $p - j$;
- y 方向： $q - l$ 。

10. 为什么 $IX = [IX, -m:-1]$

代码中：

matlab

```
1 | IX = 0:M - 1;  
2 | IX = [IX, -m:-1];
```

因为 `U2` 需要表示所有可能的索引差：

$$p - j$$

其中：

$$p = 0, 1, \dots, M - 1$$

$$j = 0, 1, \dots, m - 1$$

所以理论范围是：

$$p - j \in [-(m - 1), M - 1]$$

也就是：

$$-(m - 1), \dots, -1, 0, 1, \dots, M - 1$$

FFT 计算卷积时，数组下标按循环方式解释。长度为 $L = M + m$ 的数组中：

$$-1 \equiv L - 1$$

$$-2 \equiv L - 2$$

所以负索引要放在数组尾部。

因此构造顺序是：

$$[0, 1, \dots, M - 1, -m, \dots, -1]$$

也就是代码中的：

matlab

```
1 | IX = [Ix, -m:-1];
```

这样做的目的不是表示普通自然顺序，而是按照 FFT 循环卷积所需的索引排列方式来存放卷积核。

11. 为什么要写 $U1(n+N, m+M) = 0$

代码中：

matlab

```
1 | U1(n+N, m+M) = 0;
```

这行代码的作用是把 **U1** 补零扩展成：

$$(n + N) \times (m + M)$$

在 MATLAB 中，如果给一个超出当前矩阵大小的位置赋值，MATLAB 会自动扩展矩阵，中间新增的元素自动补 0。

因此这行代码等价于：

matlab

```
1 | U1_pad = zeros(n+N, m+M);
2 | U1_pad(1:n, 1:m) = U1;
3 | U1 = U1_pad;
```

它的目的有两个：

1. 让 `U1` 和 `U2` 的尺寸一致；
2. 避免 FFT 计算卷积时产生循环卷积混叠。

`U2` 的大小来自：

matlab

```
1 | IX = [Ix, -m:-1];  
2 | IY = [Iy, -n:-1];
```

所以：

$$\text{length}(IX) = M + m$$

$$\text{length}(IY) = N + n$$

因此 `U2` 的尺寸是：

$$(n + N) \times (m + M)$$

为了计算：

matlab

```
1 | fft2(U1).*fft2(U2)
```

`U1` 也必须扩展到相同大小。

12. 线性卷积和循环卷积

12.1 线性卷积

一维线性卷积定义为：

$$C_r = \sum_j A_j B_{r-j}$$

如果：

$$A = [a_0, a_1, a_2]$$

$$B = [b_0, b_1, b_2]$$

那么线性卷积结果长度为：

$$3 + 3 - 1 = 5$$

结果为：

$$C_0 = a_0b_0$$

$$C_1 = a_0b_1 + a_1b_0$$

$$C_2 = a_0b_2 + a_1b_1 + a_2b_0$$

$$C_3 = a_1b_2 + a_2b_1$$

$$C_4 = a_2b_2$$

12.2 循环卷积

FFT 默认计算的是循环卷积。

循环卷积可以理解为：序列的下标是一个环，超过末尾会绕回开头。

如果长度为 3 的循环卷积要容纳上面的 5 个线性卷积结果，那么：

$$C_3 \rightarrow C_0$$

$$C_4 \rightarrow C_1$$

也就是说，尾部结果会折回到前面，造成混叠。

12.3 为什么补零可以避免混叠

如果在做 FFT 卷积前把序列补零到足够长，那么线性卷积结果有足够空间存放，尾部不会绕回到前面。

在 `myFFT2` 中：

matlab

```
1 | U1(n+N, m+M) = 0;
```

就是为了让 FFT 循环卷积在需要的输出区域内等价于线性卷积。

13. 核心 FFT 卷积步骤

代码中：

matlab

```
1 | Ud1 = ifft2(fft2(U1).*fft2(U2));
```

根据卷积定理：

$$\mathcal{F}\{A * B\} = \mathcal{F}\{A\}\mathcal{F}\{B\}$$

所以：

$$A * B = \mathcal{F}^{-1}\{\mathcal{F}\{A\}\mathcal{F}\{B\}\}$$

因此这行代码是在计算二维卷积：

$$Ud1_{q,p} = \sum_l \sum_j U1_{l,j} U2_{q-l,p-j}$$

只是通过 FFT 加速实现。

14. 最后一行相位补偿

代码最后：

matlab

```
1 | Ud1 = Ud1(1:N, 1:M) .* exp(-1j*pi*(kx * kX * Ix.^2 + ky * kY * Iy.^2 + 2 *  
kX * xmin * Ix + 2 * kY * ymin * Iy + 2 * xmin * Xmin + 2 * ymin * Ymin)) *  
kx * ky;
```

它做了三件事。

14.1 取出需要的输出区域

matlab

```
1 | Ud1(1:N, 1:M)
```

只保留输出平面上的 $N \times M$ 个点。

14.2 乘输出端相位

前面拆分傅里叶核时，还有只和输出索引 p, q 有关的部分没有放入 `U1` 和 `U2`。

这些相位包括：

$$e^{-i\pi(k_x k_X p^2 + k_y k_Y q^2)}$$

以及由坐标原点引入的相位：

$$e^{-i2\pi(x_{\min} p k_X + y_{\min} q k_Y)}$$

还有常数相位：

$$e^{-i2\pi(x_{\min} X_{\min} + y_{\min} Y_{\min})}$$

它们共同对应代码中的：

matlab

```
1 | exp(-1j*pi*(kx * kX * Ix.^2 + ky * kY * Iy.^2 + 2 * kX * xmin * Ix + 2 * kY  
   | * ymin * Iy + 2 * xmin * Xmin + 2 * ymin * Ymin))
```

14.3 乘面积元

最后的：

matlab

```
1 | * kx * ky
```

对应积分离散化中的面积元：

$$dxdy \approx k_x k_y$$

15. 整个算法的逻辑流程

可以把整个函数理解成以下步骤：

1. 根据输入矩阵大小确定输入采样数量 m, n ;
2. 根据输入和输出坐标范围计算采样间隔 k_x, k_y, k_X, k_Y ;
3. 构造输入索引 j, l 和输出索引 p, q ;
4. 把原始数据 U_d 乘输入端 chirp 相位，得到 $U1$;
5. 构造依赖索引差 $p - j, q - l$ 的 chirp 卷积核 $U2$;
6. 对 $U1$ 补零，使其与 $U2$ 一样大;
7. 用 FFT 卷积定理计算 $U1$ 和 $U2$ 的二维卷积;
8. 截取需要的 $N \times M$ 输出区域;
9. 乘输出端相位和面积元 $k_x k_y$ ，得到最终结果。

16. 一句话总结

`myFFT2` 把原本的傅里叶积分离散求和：

$$\sum_l \sum_j U d_{l,j} e^{-i2\pi(x_j X_p + y_l Y_q)} k_x k_y$$

通过恒等式：

$$2jp = j^2 + p^2 - (p - j)^2$$

改写成：

$$\text{输入相位} \times \text{卷积核} \times \text{输出相位}$$

也就是：

$$U d_{q,p} = \text{输出相位} \cdot \sum_l \sum_j U1_{l,j} U2_{q-l,p-j} \cdot k_x k_y$$

其中卷积部分用 FFT 快速计算。

因此，整个函数的本质是：

用 *Bluestein/chirp - z* 思想，把任意采样范围的二维傅里叶变换转化为 *chirp* 卷积，并用 *FFT* 加速。

17. 重点记忆

后续复习时可以重点记住下面几句话：

1. $U1$ 是输入数据乘输入端 chirp 相位。
2. $U2$ 是只依赖 $p - j$ 、 $q - l$ 的 chirp 卷积核。
3. $p - j$ 是卷积中的索引差，可以理解为输出编号减输入编号。
4. $IX = [IX, -m:-1]$ 是为了按 FFT 循环卷积的顺序存放正、负索引差。
5. $U1(n+N, m+M)=0$ 是给 $U1$ 补零，让 FFT 循环卷积在所需区域内等价于线性卷积。
6. $\text{ifft2}(\text{fft2}(U1) .* \text{fft2}(U2))$ 是用卷积定理快速计算二维卷积。
7. 最后一行负责截取输出区域、补上输出端相位，并乘积分面积元。